Accelerating Evolutionary Neural Architecture Search via Multifidelity Evaluation

Shangshang Yang, Ye Tian, *Member, IEEE*, Xiaoshu Xiang, Shichen Peng, and Xingyi Zhang, *Senior Member, IEEE*

***Abstract*—Evolutionary neural architecture search (ENAS) has recently received increasing attention by effectively finding high-quality neural architectures, which however consumes high computational cost by training the architecture encoded by each individual for complete epochs in individual evaluation. Numerous ENAS approaches have been developed to reduce the evaluation cost, but it is often difficult for most of these approaches to achieve high evaluation accuracy. To address this issue, in this article, we propose an accelerated ENAS via multifidelity evaluation termed MFENAS, where the individual evaluation cost is significantly reduced by training the architecture encoded by each individual for only a small number of epochs. The balance between evaluation cost and evaluation accuracy is well maintained by suggesting a multifidelity evaluation, which identifies the potentially good individuals that cannot survive from previous generations by integrating multiple evaluations under different numbers of training epochs. Besides, a population initialization strategy is devised to produce diverse neural architectures varying from ResNet-like architectures to Inception-like ones. As shown by experiments, the proposed MFENAS takes only 0.6 GPU days to find the best architecture holding a 2.39% test error rate, which is superior to most state-of-the-art neural architecture search approaches. And the architectures transferred to CIFAR-100 and ImageNet also exhibit competitive performance.**

***Index Terms*—Convolutional neural networks (CNNs), evolutionary algorithm (EA), multifidelity evaluation, neural architecture search (NAS).**

I. Introduction

DEEP learning has made significant success in tackling various machine learning tasks, such as classification [1], object detection [2], speech recognition [3], and natural language processing [4], among many others. For achieving desirable performance, it is widely recognized that the architecture of deep neural networks (DNNs) is very crucial. For example, different types of neural networks (NNs) assist cognitive systems to deal with various types of data [5]–[7]. Searching the NNs with better performance and lower model complexity enables cognitive systems to improve the performance and simplify the design of systems for faster response to the environment. For this aim, a number of researchers are devoted to manually designing various neural architectures. Examples of this category include the ResNet [8], DenseNet [9], and GoogLeNet [10], [11]. Nevertheless, the handcraft-designed architectures often require extensive human expertise in both DNNs and the problem to be addressed, which is unrealistic for common users. This challenge arises a hot research direction in machine learning, called neural architecture search (NAS).

The NAS task was first reported by Zoph and Le [12], where the reinforcement learning (RL) was adopted as the optimization algorithm to automatically search for the architecture of convolutional NNs (CNNs). Based on the work in [12], another famous NAS approach named NASNet was developed by Zoph and Le for CNNs in [13], which can obtain neural architectures with extremely competitive performance compared to the handcraft ones. Basically, the task of NAS can be considered as the problem of combination optimization of numerous components in DNNs, which is often computationally very hard to be addressed due to the large search space. In the past years, a variety of optimization techniques have been adopted for NAS to yield high-performance architectures, including RL [14], [15], gradient optimization [16], [17], Bayesian optimization [18], [19], and evolutionary algorithm (EA) [20]–[23].

Among existing optimization techniques for NAS, EAs have attracted increasing attention due to their powerful abilities in dealing with various complex application problems [24]–[27]. The branch of NAS using EAs is usually called evolutionary

Manuscript received 7 November 2021; revised 17 February 2022 and 13 April 2022; accepted 28 May 2022. Date of publication 1 June 2022; date of current version 9 December 2022. This work was supported in part by the National Key Research and Development Project under Grant 2018AAA0100105 and Grant 2018AAA0100100; in part by the National Natural Science Foundation of China under Grant 61822301, Grant 61876123, Grant 61906001, Grant 62136008, Grant U21A20512, and Grant U1804262; in part by the Hong Kong Scholars Program under Grant XJ2019035; in part by the Anhui Provincial Natural Science Foundation under Grant 1808085J06 and Grant 1908085QF271; in part by the Collaborative Innovation Program of Universities in Anhui Province under Grant GXXT-2020-013; and in part by the State Key Laboratory of Synthetical Automation for Process Industries under Grant PAL-N201805. *(Corresponding authors: Xiaoshu Xiang; Xingyi Zhang).*

Shangshang Yang, Shichen Peng, and Xingyi Zhang are with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Artificial Intelligence, Anhui University, Hefei 230039, China (e-mail: yangshang0308@gmail.com; severus.peng@gmail.com; xyzhanghust@gmail.com).

Ye Tian and Xiaoshu Xiang are with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, Institute of Physical Science and Information Technology, Anhui University, Hefei 230601, China (e-mail: field910921@gmail.com; 94xxs13@gmail.com).

This article has supplementary material provided by the authors and color versions of one or more figures available at https://doi.org/10.1109/TCDS.2022.3179482.

Digital Object Identifier 10.1109/TCDS.2022.3179482

2379-8920 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

NAS (ENAS) and a number of ENAS approaches have been reported in the past three years [20], [21], [28]–[32]. For example, Real *et al.* [29] proposed a large-scale evolution approach to search for the architecture of DNNs by taking about 2750 GPU days [33], by which the best architecture achieved a 5.4% error rate on the CIFAR-10 data set [34]; in [30], a novel hierarchical genetic representation was devised for EAs to encode the architecture of DNNs, which can search for the architecture having a 3.75% error rate within 300 GPU days; Xie and Yuille [28] suggested an EA named Genetic-CNN to only optimize the topology of CNNs, where the best architecture found in 17 GPU days achieved a 7.1% error rate; and Sun *et al.* [32] suggested an AE-CNN algorithm to evolve CNN architectures based on two types of effective blocks and it took 27 GPU days to obtain an optimized CNN architecture with a 4.3% error rate.

In spite of the promising performance of found architectures, ENAS usually suffers from high computation cost, making it impractical for real-world applications. On the one hand, EAs are a population-based stochastic algorithm, where numerous individuals in the population need to be evaluated at each generation. On the other hand, the evaluation of each individual is computationally expensive for the task of NAS since the architecture encoded by the individual requires to be trained for complete epochs. To address this issue, a variety of approaches have been developed for ENAS, which can be roughly divided into the following two categories. The first category of approaches is developed based on the idea of reducing the number of individual evaluations in the process of optimization, where the surrogate model [35]–[38], early stopping [39], and population reduction and memory [21], [40] are three commonly used techniques. The other category of approaches focuses on reducing the computational cost of training the architecture encoded by each individual in individual evaluation, including weight inheritance [41], subset training [42], and weight sharing [43].

Nevertheless, it still poses a great challenge for existing accelerated ENAS approaches to achieve high evaluation accuracy when reducing evaluation cost, which hinders these approaches from finding high-quality architectures at low computational cost. In this article, we propose an accelerated ENAS via multifidelity evaluation, termed MFENAS, to achieve both low evaluation cost and high evaluation accuracy in NAS. The proposed MFENAS uses a small number of training epochs in individual evaluation to achieve low evaluation cost and employs multifidelity evaluation to balance evaluation cost and evaluation accuracy in NAS. The main contributions of this article are summarized as follows.

1) A novel idea of reducing the evaluation cost of individuals is suggested to accelerate ENAS, where the architecture encoded by an individual is trained for only a small number of epochs rather than complete epochs. For balancing evaluation cost and evaluation accuracy, we further propose a multifidelity evaluation to identify potentially good individuals that cannot survive from previous generations, where the idea of fidelity is introduced from computational fluid dynamics simulations [44]. The proposed multifidelity evaluation combines low-fidelity evaluation and high-fidelity evaluation that are distinguished by the number of training epochs for individual evaluation. In this case, the proposed multifidelity evaluation takes the advantages of both low computational cost in low-fidelity evaluation and high evaluation accuracy in high-fidelity evaluation, ensuring the balance between evaluation cost and evaluation accuracy. Hence, the proposed multifidelity evaluation can facilitate ENAS in achieving high-quality architectures at low computational cost.
2) Based on the proposed multifidelity evaluation, a multiobjective EA called MFENAS is developed in the framework of NSGA-II [45] for fast NAS. The validation error rate and model complexity are adopted as two optimization objectives with the aim of finding high-accuracy neural architectures under different levels of complexity. For high diversity of initial neural architectures, a population initialization strategy is suggested to produce a set of neural architectures varying from Inception-like architectures to ResNet-like ones. For maintaining the diversity of neural architectures during NAS, a variable-length integer vector is utilized to encode neural architectures, and a genetic operator is devised to equip with variable-length individuals in the proposed MFENAS.
3) The performance of the proposed MFENAS is verified on three widely used data sets, namely, CIFAR-10, CIFAR-100, and ImageNet. The search of neural architectures is implemented on CIFAR-10 and the architectures found by the proposed MFENAS are evaluated on CIFAR-10, CIFAR-100, and ImageNet. The best architecture found by the proposed MFENAS achieves a 2.39% test error rate on CIFAR-10 at the cost of only 0.6 GPU days on one NVIDIA 2080TI GPU, which is superior over existing EA and non-EA-based NAS approaches. The best architecture is transferred to CIFAR-100 and ImageNet, which also holds a promising performance in comparison with existing NAS approaches.

The remainder of this article is organized as follows. Section II presents preliminaries and related work on ENAS. Section III elaborates the proposed MFENAS and Section IV reports the experimental results for verifying the performance of the proposed MFENAS. Finally, conclusions and future work are given in Section V.

II. Preliminaries and Related Work

In this section, we first briefly introduce some preliminaries of ENAS. Then, we review existing work on reducing the computation cost of ENAS. Finally, we present the motivation of this study.

A. Evolutionary Neural Architecture Search

For the enormous demand for small-sized networks in real-world applications, both the accuracy and the model complexity of architectures are considered in the searching [20], [46], thus NAS can be formulated as the following biobjective

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

optimization problem:

$$\min_{\mathbf{x}} \mathbf{F}(\mathbf{x}) = \begin{cases} f_1(\mathbf{x}) = f_{\text{val_error}}(\mathbf{x}, D_{\text{val}}) \\ = \frac{\sum_{i=1}^{||D_{\text{val}}||} \text{Error}(\mathbf{x}, D_{\text{val}}^i)}{||D_{\text{val}}||} \\ f_2(\mathbf{x}) = f_{\text{complexity}}(\mathbf{x}) \end{cases} \tag{1}$$

where $\mathbf{x}$ represents the architecture to be optimized, $||D_{\text{val}}||$ denotes the number of samples in validation data set D_{val}, $\text{Error}(x, D_{\text{val}})$ represents the ith sample, D_{val}^i is mistakenly classified by the network $\mathbf{x}$, thus $f_1(\mathbf{x})$ is the classification error rate of $\mathbf{x}$ on validation data set D_{val}. $f_2(\mathbf{x})$ denotes the model complexity of $\mathbf{x}$. The model complexity can be measured by the number of model parameters, the number of floating-point operations, and inference time [20], [46]. In this article, we adopt the biobjective optimization problem formulation for NAS, where the number of model parameters is used to measure the model complexity.

To address multiobjective optimization problems, a variety of approaches have been developed based on different ideas, e.g., Bayesian optimization [47], simulated annealing [48], Tabu search [49], and EAs [50]. Among these approaches, EAs are the most widely investigated approaches to multiobjective optimization due to the fact that they maintain a population approximating the set of optimal solutions. A large number of EAs have been developed to solve multiobjective optimization problems in different research areas, such as community detection [25], shelter location problem [26], and vehicle routing problem [51].

Generally, an EA consists of five main steps as follows.

1) Population initialization, by which an initial population is created at the beginning of EA and each individual in the population encodes a solution of the problem to be solved.
2) Mating pool selection, by which individuals are mated to select parent individuals.
3) Genetic operation, where crossover and mutation operators are used to create offspring individuals from the parent individuals.
4) Fitness evaluation, where the quality of each individual in the population is measured.
5) Environment selection, by which good individuals in the parent and offspring population survive for the next generation. It is worth noting that the fitness evaluation of each individual for NAS is achieved by training the architecture encoded by each individual for complete epochs on the training data set, which is often characterized with expensive computational cost.

B. Related Work

To reduce the computational cost of ENAS, many approaches have been developed and can be roughly divided into the following two categories.

1) Reducing the Number of Individuals to Be Evaluated in Real: Three ideas are often adopted in existing ENAS to reduce the number of individuals to be evaluated in real, namely, surrogate model, early stopping, population reduction, and memory.

Surrogate Model: The main idea of surrogate model-based ENAS approaches is to predict the quality of architectures encoded by some individuals by surrogate models to reduce the number of individuals to be evaluated in real. The representatives belonging to this idea includes [38], [52]. Among these, Sun *et al.* [38] adopted the random forest as the surrogate model to predict the quality of newly created individuals at each generation of EAs. Experimental results indicated that the surrogate model-assisted ENAS saved 68.18% and 69.70% computational cost on CIFAR10 and CIFAR100, respectively. Nevertheless, the best architecture found on CIFAR10 can still achieve 94.70% test accuracy, which is only 0.06% worse than the version without a surrogate model. There also exist some surrogate-assisted NAS approaches which are not based on the framework of EAs [35], [37].

Early Stopping: The idea of early stopping is to terminate the training of poor individuals based on the performance of architectures in the early training stage to reduce the number of individuals to be evaluated. Suganuma *et al.* [53] developed an early stopping strategy based on a reference curve to accelerate ENAS, where the reference curve is updated on the basis of the best individual in the population at each generation. A similar idea is also adopted for accelerating ENAS in [54] and [55]. Despite that the idea of early stopping can considerably reduce the computational cost of ENAS approaches, it easily leads to inaccurate estimation on individual quality, especially for complicated architectures as stated in [56]. This makes it hard for early stopping-based ENAS approaches to achieve competitive performance in NAS.

Population Reduction and Memory: The population reduction achieves the accelerating of ENAS by recursively reducing the size of population [40], [57], whereas the population memory avoids evaluating the same individuals that have been evaluated in the evolution of the population to solve this task [21], [58]. Fan *et al.* [57] divided the whole optimization process of the ENAS approach into different stages with different population sizes, where a large population is used to ensure the global search ability in the early stage and a small population is adopted to reduce the number of individuals to be evaluated in the population in the later stage. Sun *et al.* [21] utilized a hashing method to record both architecture information and fitness of each individual, so that the fitness value for an individual can be directly obtained if this individual has been recorded. Despite avoiding redundant individual evaluations in ENAS, these strategies still consume expensive computational cost to achieve high-quality architectures [56].

2) Reducing the Evaluation Cost of Each Individual: The subset training, weight inheritance, and weight sharing are three ideas that are widely adopted in ENAS approaches to reduce the evaluation cost of each individual.

Subset Training: The subset training is to train the architecture on a small subset selected from the original data set having a large number of data, by which the computation cost on evaluating each individual can be effectively reduced. Following this idea, Liu *et al.* [42] suggested an ENAS approach to search for the optimal architecture for medical image denoising. To reduce the training time without seriously degrading

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

the performance, a small subset having similar properties to those in the original data set is randomly selected for exploring promising CNN architectures. Despite that the subset training can effectively reduce the computational cost of ENAS, there exists a big challenge of overfitting problem due to the difficulty of properly selecting a subset.

Weight Inheritance and Weight Sharing: The weight inheritance is a technique that the architectures encoded by offspring individuals inherit most weights in the architectures encoded by parent individuals, since the genetic operators in ENAS produce offspring individuals by inheriting most architectures encoded by parent individuals, where the representative methods include [41] and [59]. In contrast to extra training needed for the evaluation of weight inheritance, the evaluation of weight sharing is directly based on the weights of the architecture encoded by each individual obtained from a set of weights stored in a SuperNet model. Hence, the individuals in ENAS can be evaluated by training only one SuperNet model instead of training all the architectures encoded by these individuals, which thus considerably reduces the computational cost of ENAS, the representative methods include [14] and [43].

C. Motivation

Different from existing accelerated ENAS approaches, the proposed MFENAS reduces the evaluation cost of each individual by means of training the architecture encoded by each individual for only a small number of epochs rather than complete epochs. This idea is motivated from the observation that the performance ranking of individuals at each generation will not change considerably no matter whether the architectures encoded by the individuals are trained for a small number of epochs or complete epochs. To illustrate this fact, Fig. 1 shows the profiles of performance ranking on 1000 architectures under different numbers of training epochs, where 1000 neural architectures are randomly sampled in the cell-based search space according to the setting of training suggested in [13], [17], and [20], and the number of nodes in each cell ranges from 5 to 12. Specifically, Fig. 1(a) presents the validation accuracy of ten architectures that are randomly selected from the 1000 architectures, where the architectures having high accuracy under a small number of training epochs also hold high accuracy under complete epochs and vice versa.

This observation also often holds when the accuracy difference between two architectures is not significant as depicted in Fig. 1(b), which gives the profile of the Kendall Tau rank correlation coefficient (Kendall's τ for short) between the performance ranking at each epoch and that of the final epoch on all of the 1000 randomly sampled architectures. Kendall's τ is a widely used indicator to measure the correlation between two different rankings of items [60]. $\tau \in [-1, 1]$, where $\tau = 1$ means the two rankings are exactly the same and $\tau = -1$ means they are completely opposite.

From Fig. 1(b), we can find the following two results. On the one hand, the performance ranking of architectures at any number of training epochs holds a high correlation with the ranking at final epoch, where Kendall's τ value is larger than 0.45 and competitive in comparison with most of the surrogate models used in ENAS [35], [37], [38]. On the other hand, Kendall's τ value increases considerably as the number of training epochs increases, which inspires us to consider the idea of multifidelity optimization (MFO) [44] to ensure the performance of ENAS under a small number of epochs. It is worth noting that the MFO is a popular idea for striking the balance between the effectiveness and efficiency of an algorithm by means of combining high fidelity and low fidelity, where the high fidelity leads to high-accuracy evaluation but low computation efficiency, and the low fidelity leads to high computation efficiency but low-accuracy evaluation [61].

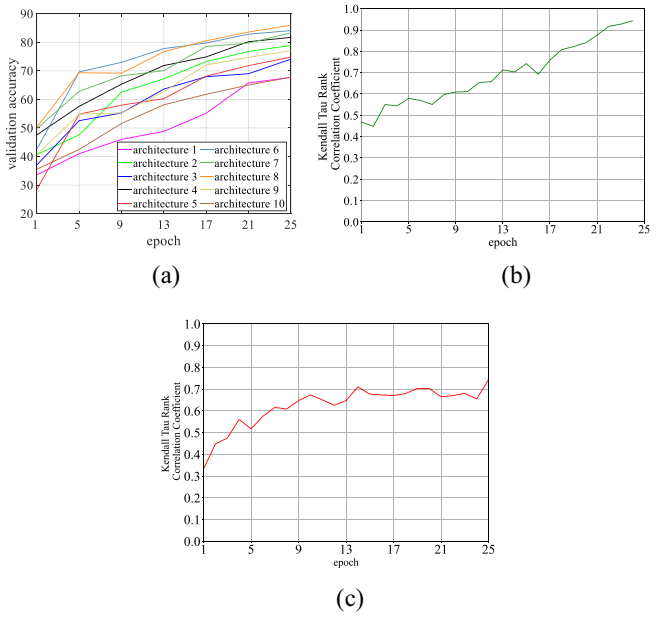


Fig. 1. Performance ranking observation on 1000 randomly sampled architectures in cell-based architecture search space. It is worth noting that the number of complete epochs for training models is set to 25 according to NASNet [13] and NSGA-Net [20]. (a) Validation accuracy of ten randomly selected architectures. (b) Kendall Tau rank correlation coefficient on 1000 architectures randomly sampled from our search space. (c) Kendall Tau rank correlation coefficient on 1000 architectures randomly selected from the NAS-bench-201 benchmark.

It is noteworthy the fitness at a small number of epochs may not always be correlated with the fitness at a large number of epochs for some architectures sampled from the NASNet search space [13]. Fig. 1(c) illustrates this fact by presenting Kendall's τ value between performance ranking under 200 training epochs and that under different numbers of epochs ranging from 1 to 25 in the NAS-bench-201 benchmark. According to Fig. 1(c), the correlation between performance ranking under 200 training epochs and that under 25 epochs is smaller than 0.8. Nevertheless, in this article, we assume that there also exist many architectures whose fitness at a small number of epochs is correlated with that at a large number of epochs. A similar assumption has been widely adopted for architecture evaluation in most NAS approaches, since it is indeed impractical in architecture search algorithms to afford the highly expensive cost if each architecture is trained for hundreds of epochs. In spite of losing a little evaluation accuracy in search, the existing NAS approaches have verified the effectiveness on training networks for a small number of

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FORMAL NOTATIONS USED IN THIS ARTICLE

Notation	Description
$\mathbf{P}$ ($\mathbf{Q}$)	a population $\mathbf{P} = \{\mathbf{P_1}, \mathbf{P_2}, \cdots, \}$ (a offspring population)
$\mathbf{P'}$	a parent population for generating offsprings
$\mathbf{P^S}$ ($\mathbf{P^E}$)	a population surviving (eliminated) from selection
$\mathbf{E}$	an archive storing potentially good individuals
I ($\mathbf{P}_i$)	the individual (the i-th individual in $\mathbf{P}$) $I = \mathbf{P_i} = \{\mathbf{NC}, \mathbf{RC}\}$
$\mathbf{NC}$ ($\mathbf{RC}$)	a normal cell (a reduction cell) $\mathbf{NC}(\mathbf{RC}) = (\mathbf{n_0}, \mathbf{n_1}, \cdots)$
$\mathbf{n_i}$	the i-th node in $\mathbf{NC}(\mathbf{RC})$, $\mathbf{n_i} = (\mathbf{L}, Op)$
$\mathbf{L}$	a vector denotes the link information
Op	a number refers to used operation
Gen, Pop	maximum number of generations, Population size
$Node_{range}$	range of the number of nodes in initialization
MF	number of fidelities for multi-fidelity evaluation
S	current number of epoch for training process
$SS_{counter}$	count of surviving according to one evaluation
$MS_{counter}$	count of surviving according to multi-fidelity evaluation
$ME_{counter}$	count of being eliminated according to multi-fidelity evaluation
H	all previous outputs before node k $H = \{\hat{h}[0], \hat{h}[1], h[0], ..., h[k-1]\}$
$\hat{h}[0]$, $\hat{h}[1]$	the outputs of two previous cells
$h[k]$, in_i	the output of node k, the i-the input of a node

epochs [20], [23], [30], [62], [63]. Take the NSGA-Net [20] as an example, each architecture evaluation employs only 25 training epochs for the architecture to finally achieve an architecture with as low as 2.75% test error rate after evolution, which is competitive to the NAS approaches [13], [31] based on training neural architectures for hundreds of epochs. Therefore, the proposed MFENAS is also based on such an assumption and suggests a multifidelity evaluation strategy to further reduce the number of training epochs for improving the computational efficiency of NAS approaches.

Following the above idea, we propose an accelerated ENAS approach via multifidelity evaluation, named MFENAS, where each individual is evaluated by training the architecture encoded by each individual for only a small number of epochs. For balancing evaluation cost and evaluation accuracy, we suggest a multifidelity evaluation combining high-fidelity evaluation and low-fidelity evaluation, where high-fidelity evaluation indicates individual evaluation under a large number of training epochs, and low-fidelity evaluation indicates that under a small number of training epochs. Hence, the multifidelity evaluation can identify potentially promising individuals that cannot survive from previous generations by integrating multiple evaluations under different numbers of training epochs.

III. PROPOSED MFENAS

In this section, we first present the overall framework of the proposed MFENAS. Then, we elaborate the multifidelity evaluation in the proposed MFENAS. Finally, further details in the proposed MFENAS are presented.

A. Overall Framework of MFENAS

To start with, the formal notations used in this article are listed in Table I.

To simultaneously optimize the accuracy and complexity of the neural architecture in NAS, the proposed MFENAS employs NSGA-II as the optimizer due to its high competitiveness in solving biobjective optimization problems [64].

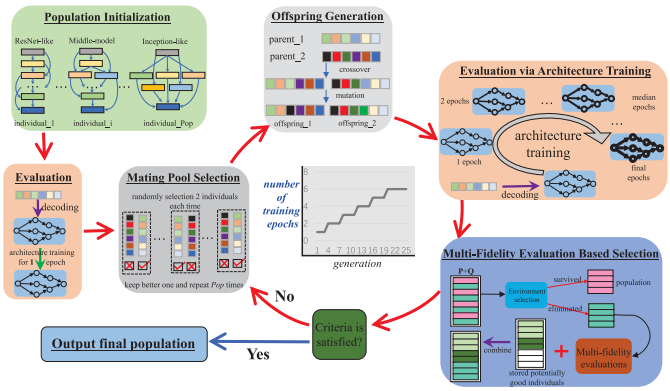


Fig. 2. Overall framework of the proposed MFENAS.

Compared with existing work on NSGA-II for NAS, the proposed MFENAS is characterized with two key aspects: 1) a small number of architecture training epochs for individual evaluation and 2) a multifidelity evaluation for environment selection at each generation. Specifically, Fig. 2 presents the overall framework of MFENAS, which mainly consists of six steps.

First, a well-distributed population is initially generated by producing diverse neural architectures that vary from ResNet-like architectures to Inception-like ones. Second, the individuals in the generated population are efficiently evaluated by training the neural architectures encoded by these individuals for a certain number of epochs, which is initially set to one. Third, binary tournament selection is applied to the generated population to mate individuals in the population. Fourth, an offspring population is produced from the parent population that is generated at the last step by implementing revised single-point crossover and mutation (described in Section III-C3) in mated individuals of the parent population. Fifth, the individuals in the offspring population are efficiently evaluated by training the neural architectures encoded by these individuals for a certain number of epochs, which will be updated according to the generation and the number of training epochs. Finally, a multifidelity evaluation-based selection is used to select individuals from the union of parent population and offspring population, where the multifidelity evaluation identifies and maintains potentially good individuals that are eliminated by the environment selection in NSGA-II [45]. The second to the sixth step will repeat until the evolution termination criteria are satisfied, after which the nondominated individuals will be output. For more details, Algorithm 1 presents the main steps of the proposed MFENAS.

B. Proposed Multifidelity Evaluation

In the proposed MFENAS, it is expected that the evaluation cost of individuals can be significantly reduced by training the architecture encoded by each individual for only a small number of epochs, which will greatly accelerate the entire ENAS process. Nevertheless, the evaluation cost reduction hinders the architecture training from achieving high accuracy in the individual evaluation, and thus leads to inappropriate elimination of good individuals in the environment selection

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Main Steps of MFENAS

Input: *Gen*: Maximum number of generations; *Pop*: Population size; $Node_{range}$: Range of the number of nodes in initialization; *MF*: Number of fidelities for multi-fidelity evaluation;
Output: **P**: Population;
1: $\mathbf{P}, \mathbf{E}, S \leftarrow Initialization(Pop, Node_{range}), \emptyset, 1$;
2: Evaluate individuals in **P** by training the architectures encoded by the individuals for S epoch and compute the fitness by (1);
3: **for** $g = 1$ to *Gen* **do**
4: $\mathbf{P}' \leftarrow$ Select *Pop* parents according to the non-dominated front number and crowding distance of individuals in **P**; % Mating pool selection
5: $\mathbf{Q} \leftarrow Genetic\ Operator(\mathbf{P}')$;
6: Evaluate individuals in **Q** by training the architectures encoded by the individuals for S epoch(s) and compute the fitness by (1);
7: $\mathbf{P}, \mathbf{E}, S \leftarrow$ *Multi-Fidelity Evaluation Based Selection* $(\mathbf{P}, \mathbf{Q}, \mathbf{E}, g, MF, S)$; % Algorithm 2
8: **end for**
9: **return** **P**;

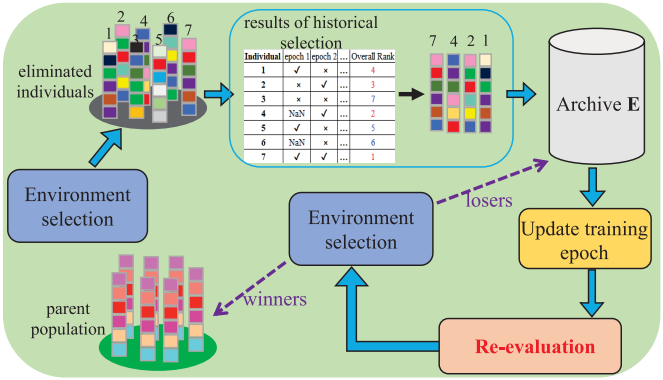


Fig. 3. Main procedure of multifidelity evaluation-based selection.

[38], [41], [42]. To address this issue, we propose a multifidelity evaluation-based selection for maintaining potentially good individuals that cannot survive from the current environment selection by multiple evaluations under different numbers of training epochs for each of these individuals.

The proposed multifidelity evaluation-based selection mainly consists of five steps as shown in Fig. 3. At each generation of ENAS, the environment selection of NSGA-II is first implemented to select a set of individuals from parent and offspring populations as mentioned in Section III-A, and to eliminate the remaining individuals in the two populations. Second, the potentially good individuals are selected from the eliminated individuals by multifidelity evaluation, which are stored in an archive that includes previous potentially good individuals and their survival or elimination information in previous environment selection. Third, the number of training epochs updates according to the generation and the number of training epochs. Fourth, the updated number of training epochs is adopted in training architectures encoded by the individuals in the archive and those surviving in the environment selection, and thus these individuals continue to be re-evaluated based on their model parameters obtained from previous training. Finally, the environment selection in NSGA-II is applied to the re-evaluated individuals to maintain some good individuals as the parent population at the next generation, while the remaining individuals in the selection will be stored in the archive.

Algorithm 2 Multifidelity Evaluation-Based Selection $(\mathbf{P}, \mathbf{E}, \mathbf{E}', g, MF, S)$

Input: **P**: Population; **Q**: Offspring; **E**: Archive; g: Current generation; *MF*: Number of fidelities for multi-fidelity evaluation; S: Current number of training epochs;
Output: **P**: Population; **E**: Archive;
1: $\mathbf{P^S}, \mathbf{P^E} \leftarrow$ Select *Pop* individuals from $\mathbf{P}+\mathbf{Q}$ by the environment selection of NSGA-II;
2: $SS_{counter}$ +1 for individuals in $\mathbf{P^S}$, $ME_{counter}$ +1 for individuals in $\mathbf{P^E}$ whose $ME_{counter} == 0$;
3: $\mathbf{T} \leftarrow$ Sort the individuals in **E** and $\mathbf{P^E}$ using four criteria;
4: $\mathbf{E} \leftarrow$ *Truncated operation* (**T**);
5: **if** $mod(g, \lfloor \frac{Gen}{MF} \rfloor) == 0$ and $S \neq MF$ **then**
6: $S \leftarrow S + 1$;
7: Continue training architectures encoded by individuals in $\mathbf{P^S} + \mathbf{E}$ for S epochs to re-evaluate these individuals by (1);
8: $\mathbf{P}, \mathbf{E} \leftarrow$ Select *Pop* individuals from $\mathbf{P^S} + \mathbf{E}$ by the environment selection of NSGA-II;
9: $MS_{counter}$ +1, $SS_{counter}$ = 0 for individuals in **P**, $MS_{counter}$ +1 for individuals in **E** whose $ME_{counter} == 0$, $ME_{counter}$ +1 for individuals in **E**;
10: **else**
11: $\mathbf{P} \leftarrow \mathbf{P^S}$;
12: **end if**
13: **if** $g ==$ *Gen* **then**
14: Continue training architectures encoded by individuals in $\mathbf{P^S} + \mathbf{E}$ for complete epochs to get final validation error and number of model parameters in (1);
15: $\mathbf{P}, \mathbf{E} \leftarrow$ Select *Pop* individuals from $\mathbf{P^S} + \mathbf{E}$ by the environment selection of NSGA-II;
16: **end if**
17: **return** **P**, **E**, S;

As can be seen, the proposed multifidelity evaluation is devoted to modifying the performance indicator of the architectures as evolution proceeds. Specifically, the evaluation results under the 1st training epoch are first utilized for distinguishing neural architectures at the 1st generation. This indicator under the 1st epoch will be used for several generations until the population performance cannot be improved; otherwise, the evaluation under the 2nd epoch will then be employed as the indicator of population performance. This indicator under the 2nd epoch will also be used for several generations until the population performance cannot be improved. The number of training epochs will increase by 1 each time when the performance indicator needs to be adapted, and the above procedure will be repeated until the maximum number of generations is achieved.

Algorithm 2 summarizes the detailed steps of the proposed multifidelity evaluation-based selection. To begin with, the environment selection of NSGA-II is applied to the union of parent population **P** and offspring population **Q**, where surviving individuals and eliminated individuals in the selection are denoted as $\mathbf{P^S}$ and $\mathbf{P^E}$, respectively (step 1).

Then, a multifidelity evaluation is employed to select potentially good individuals from $\mathbf{P^E}$ by making use of individual selection under different numbers of epochs for training the architectures encoded by the individuals. To record the individual selection under different numbers of training epochs, we devise three counters for each individual in **P** and **Q**.

1) The SS_{counter} denotes the count of surviving according to one evaluation.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

2) The MS_{counter} denotes the count of surviving according to multifidelity evaluation.
3) The ME_{counter} denotes the count of being eliminated according to multifidelity evaluation.

Based on the environment selection, the SS_{counter} adds one for each individual in $\mathbf{P^S}$, and the ME_{counter} adds one for each individual in $\mathbf{P^E}$ whose ME_{counter} is zero (step 2).

Based on the three counters, four criteria are suggested to sort the individuals in the union of $\mathbf{E}$ and $\mathbf{P^E}$ (step 3)

$$\begin{cases} MS_{\text{counter}} - ME_{\text{counter}} \\ -(MS_{\text{counter}} + ME_{\text{counter}}) \\ SS_{\text{counter}} \\ f_2(\mathbf{x}) \end{cases}$$

where the first criteria hold the highest priority and aim to retain the individuals that frequently survive according to multifidelity evaluation, the second criteria hold the second highest priority and tend to maintain the individuals that rarely experience evaluations under different fidelities, the third criteria hold the third highest priority and are prone to keep the individuals that frequently survive according to one evaluation, and the fourth criteria hold the fourth highest priority and prefer the individuals that are encoded from large and complex neural architectures. The individuals in the union of $\mathbf{E}$ and $\mathbf{P^E}$ are sorted according to the four criteria. Next, a truncated operation is used to select the former $|\mathbf{E}|$ highest-criterion individuals and store these individuals to the archive $\mathbf{E}$, where $|\mathbf{E}|$ denotes a predefined size of $\mathbf{E}$ (step 4).

To further evaluate the individuals in archive $\mathbf{E}$, we periodically update the number of training epochs S for the architectures encoded by the individuals by adding 1 to S for every $\lfloor(\text{Gen}/MF)\rfloor$ generations, where Gen is the maximum number of generations and MF is the number of fidelities for multifidelity evaluation (also denoting the maximum number of training epochs for each individual evaluation). With the updated number of training epochs S, the individuals in both archive $\mathbf{E}$ and population $\mathbf{P^S}$ are re-evaluated by continuing training the architectures encoded by these individuals for S epochs based on model parameters obtained from previous training (step 7).

After the re-evaluation, the environment selection of NSGA-II is applied to the union of $\mathbf{P^S}$ and $\mathbf{E}$ to obtain a surviving population for a new parent population and an eliminated population stored in the archive $\mathbf{E}$ at the next generation (step 8). The three counters are then updated according to environment selection: the MS_{counter} adds one, the SS_{counter} is set as zero for each individual in $\mathbf{P}$, the MS_{counter} adds one for the individuals that are stored in archive $\mathbf{E}$ for the first time, and the ME_{counter} adds 1 for each individual in archive $\mathbf{E}$ (step 9).

When it achieves the maximum number of generations, all architectures encoded by the individuals in the union of $\mathbf{P^S}$ and $\mathbf{E}$ continue to be trained for complete epochs to get their final fitness values in (1) (step 14). Based on the fitness values, the environment selection of NSGA-II is used to acquire a final population consisting of nondominated individuals (step 15).

Note that the number of fidelities for multifidelity evaluation MF refers to how many different types of evaluations in the proposed MFENAS, and one type of evaluation indicates an individual evaluation under a specific number of training epochs. For example, $MF = 1$ indicates that only one type of evaluation is used for each individual, i.e., architecture training for one epoch. When $MF = 3$, it indicates that three types of evaluations can be used for one individual, i.e., architecture training for one epoch, two epochs, and three epochs. A large value of MF leads to an accurate individual evaluation, while a small value of MF leads to an efficient individual evaluation for the proposed MFENAS. Hence, MF plays an important role in striking a balance between evaluation efficiency and accuracy, which will be discussed in the experiments of Section IV-D.

C. Related Details

In this section, we give further details of the proposed MFENAS, including architecture search space, encoding strategy, genetic operator, and population initialization strategy.

1) Architecture Search Space for NAS: In our approach, the cell-based architecture search space [13] is adopted to build the whole network, which consists of a stack of several cells, including normal cells and reduction cells. Due to the limit of pages, the details about our used architecture search space are summarized in the supplementary material.

2) Encoding Strategy: In the proposed MFENAS, a neural architecture is encoded by an individual $I = \{\mathbf{NC}, \mathbf{RC}\}$, where two vectors $\mathbf{NC}$ and $\mathbf{RC}$ represent normal cell and reduction cell, respectively. Each of the two vectors is composed of some subvectors, where each of these subvectors represents a node. Specifically, $\mathbf{NC}$ and $\mathbf{RC}$ are denoted as follows:

$$\begin{cases} \mathbf{NC} = (\mathbf{n_0}, \mathbf{n_1}, \ldots, \mathbf{n_k}, \ldots, \mathbf{n_{N-1}}), k = 0, 1, \ldots, \mathbf{N-1} \\ \mathbf{RC} = \left(\tilde{\mathbf{n}_0}, \tilde{\mathbf{n}_1}, \ldots, \tilde{\mathbf{n}_m}, \ldots, \tilde{\mathbf{n}_{R-1}}\right), m = 0, 1, \ldots, \mathbf{R-1} \end{cases} \tag{2}$$

where the subvector $\mathbf{n_k}$ ($\tilde{\mathbf{n}_m}$) denotes the node k (node m) in the normal cell $\mathbf{NC}$ (reduction cell $\mathbf{RC}$), and $\mathbf{N}$ ($\mathbf{R}$) denotes the number of subvectors in $\mathbf{NC}$ ($\mathbf{RC}$). Each subvector consists of an operation in the corresponding node and a set of links for connecting to some precursor nodes of this node. Specifically, the subvector $\mathbf{n_k}$ can be denoted as follows:

$$\begin{aligned} \mathbf{n_k} = (\mathbf{L}, Op) = \left(\widehat{l}_0, \widehat{l}_1, l_0, l_1, \ldots, l_i, \ldots, l_{k-1}, Op\right), \widehat{l}_0, \widehat{l}_1 \in \{0, 1\} \\ i = 0, 1, \ldots, k-1,\ l_i \in \{0, 1\},\ Op \in \{0, 1, \ldots, 10\} \end{aligned} \tag{3}$$

where $\mathbf{L} = (\widehat{l}_0, \widehat{l}_1, l_0, l_1, \ldots, l_i, \ldots, l_{k-1})$ is a vector of links in the node k, $\widehat{l}_0$ ($\widehat{l}_1$) indicates whether the node k links to the first (second) input node, l_i indicates whether the node k links to the node i, and Op denotes the number of a specific operation for the node k. Due to the limit of pages, the details about all candidate operations are summarized in the supplementary material.

For better understanding, Fig. 4 gives an illustrative example of the proposed encoding strategy. As can be seen from Fig. 4, the encoding $\mathbf{NC}$ with seven subvectors represents a normal cell consisting of seven nodes, where each subvector represents a node with specific links and operation. For example, the node 4 encoded by $(0, 1, 1, 1, 0, 0, 3)$ indicates that the node 4 links to the second input node $\widehat{h}[1]$, the node 0, the node 1, and adopts the operation Conv $1 * 3 + 3 * 1$.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

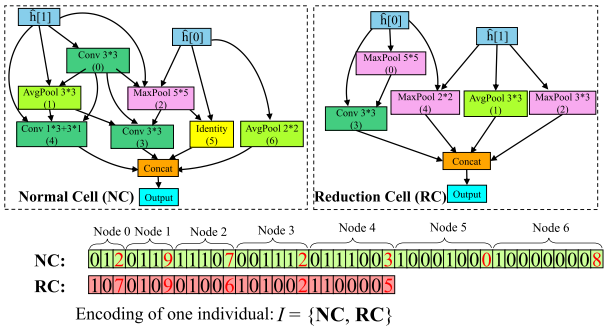


Fig. 4. Illustrative example of the proposed encoding strategy.

3) Genetic Operator: Based on the suggested architecture search space, we specially design an effective genetic operator consisting of crossover and mutation for offspring generation in the proposed MFENAS.

Crossover: The crossover operator consists of two components: 1) the intercell crossover and 2) the intracell crossover. The intercell crossover is to exchange normal cells or reduction cells between two parent individuals. For example, given two parent individuals $\mathbf{P_1} = \{\mathbf{NC_1}, \mathbf{RC_1}\}$ and $\mathbf{P_2} = \{\mathbf{NC_2}, \mathbf{RC_2}\}$, two offspring individuals can be obtained by

$$\begin{aligned} \mathbf{P_1} &= \{\mathbf{NC_1}, \mathbf{RC_1}\} \\ \mathbf{P_2} &= \{\mathbf{NC_2}, \mathbf{RC_2}\} \end{aligned} \Rightarrow \begin{aligned} \mathbf{Q_1} &= \{\mathbf{NC_1}, \mathbf{RC_2}\} \\ \mathbf{Q_2} &= \{\mathbf{NC_2}, \mathbf{RC_1}\} \end{aligned} \tag{4}$$

The intracell crossover is to exchange links and operations in normal cells or reduction cells between two parent individuals. Specifically, the intracell crossover is executed based on a single-point crossover or a revised one-way crossover. Take the following two normal cells $\mathbf{NC_1}$ and $\mathbf{NC_2}$ as an example:

$$\begin{cases} \mathbf{NC_1} = \Big(\mathbf{n_0^1}, \mathbf{n_1^1}, \ldots, \mathbf{n_k^1} = \big(\widehat{l}_0^1, \widehat{l}_1^1, \ldots \\ \qquad l_i^1, \ldots, l_{k-1}^1, Op^1\big), \ldots, \mathbf{n_{N_1-1}^1}\Big), \ N_1 \text{ nodes} \\ \mathbf{NC_2} = \Big(\mathbf{n_0^2}, \mathbf{n_1^2}, \ldots, \mathbf{n_k^2} = \big(\widehat{l}_0^2, \widehat{l}_1^2, \ldots \\ \qquad l_i^2, \ldots, l_{k-1}^2, Op^2\big), \ldots, \mathbf{n_{N_2-1}^2}\Big), \ N_2 \text{ nodes} \end{cases} \tag{5}$$

where the length of $\mathbf{NC_1}$ and $\mathbf{NC_2}$ is $([(N_1+5)N_1]/2)$ and $([(N_2+5)N_2]/2)$, respectively. Suppose $N_1 \le N_2$ and a random integer RI is sampled from 0 to $([(N_2+5)N_2]/2)$, then two offspring cells $\mathbf{NC_1^o}$ and $\mathbf{NC_2^o}$ can be generated by a single-point crossover [65]

$$\text{when RI} \ \le \ \frac{(\mathrm{N_1}+5)\mathrm{N_1}}{2} \Rightarrow \begin{cases} \mathbf{NC_1}[1:RI] \Leftrightarrow \mathbf{NC_2}[1:RI] \\ \mathbf{NC_1^o} \longleftarrow \mathbf{NC_1} \\ \mathbf{NC_2^o} \longleftarrow \mathbf{NC_2} \end{cases} \tag{6}$$

or a revised one-way crossover

$$\text{when} \ \frac{(\mathrm{k}+5)\mathrm{k}}{2} < \mathrm{RI} \ \le \ \frac{(\mathrm{k}+6)(\mathrm{k}+1)}{2} \Rightarrow \begin{cases} \mathbf{n_k^{2\to 1}} = \ \big(\mathbf{n_k^2}[1:N_1+2], \mathbf{n_k^2}[\text{end}]\big) \\ \qquad \text{where } \mathrm{N_1}-1 < \mathrm{k} \le \mathrm{N_2}-1 \\ \mathbf{NC_1^o} \longleftarrow \big(\mathbf{NC_1}, \mathbf{n_k^{2\to 1}}\big) \\ \mathbf{NC_2^o} \longleftarrow \mathbf{NC_2} \end{cases} \tag{7}$$

where $\mathbf{NC_1}[1:RI]$ denotes the subvector clapped from the first bit to the RIth bit in $\mathbf{NC_1}$, and $\mathbf{n_k^2}[\text{end}]$ is the last bit of $\mathbf{n_k^2}$.

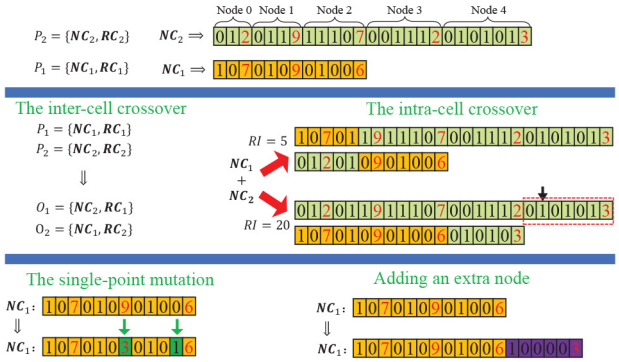


Fig. 5. Illustrative example of genetic operator for offspring generation, where only the information of normal cells is presented for simplicity.

$RI \le ([(N_1+5)N_1]/2)$ in (6) ensures that the crossover point RI locates in one bit of the shorter parent cell between the two parent cells. Then, $\mathbf{NC_1}[1:RI]$ will be exchanged with $\mathbf{NC_2}[1:RI]$ using a single-point crossover.

$([(k+5)k]/2) < RI \le ([(k+6)(k+1)]/2)$ in (7) indicates that RI locates in the subvector denoting node k. Then, a revised one-way crossover is designed to identify useful bits in the node k and copy the identified bits to $\mathbf{NC_1}$. As presented in (7), the links $\mathbf{n_k^2}[1:N_1+2]$ and the operation $\mathbf{n_k^2}[\text{end}]$ in the node k are identified as useful bits for $\mathbf{NC_1}$, and thus are copied from $\mathbf{NC_2}$ to $\mathbf{NC_1}$.

Mutation: The mutation operator also consists of two parts: 1) a single-point mutation and 2) an operator of adding an extra node. Note that the mutation probability for links and that for operations are different in the single-point mutation. The operator of adding an extra node to $\mathbf{NC_1}$ can be denoted by

$$\begin{cases} \mathbf{n_{N1}^{mutation}} \longleftarrow \text{Random}(\mathbf{L}, \text{Op}) \\ \mathbf{NC_1} \longleftarrow \big(\mathbf{NC_1}, \mathbf{n_{N1}^{mutation}}\big) \end{cases} \tag{8}$$

where $\mathbf{n_{N1}^{mutation}}$ is a new node generated in a random way and is then combined with cell $\mathbf{NC_1}$ into a new $\mathbf{NC_1}$.

For better understanding, Fig. 5 gives an illustrative example of the genetic operator for offspring generation in the proposed MFENAS. Given two parent individuals $P_1 = \{\mathbf{NC_1}, \mathbf{RC_1}\}$ and $P_2 = \{\mathbf{NC_2}, \mathbf{RC_2}\}$, one of the intercell crossover and the intracell crossover is randomly selected and applied to the two individuals. The intercell crossover exchanges cells $\mathbf{NC_1}$ and $\mathbf{NC_2}$ in the two parent individuals to obtain two offspring individuals $O_1 = \{\mathbf{NC_2}, \mathbf{RC_1}\}$ and $O_2 = \{\mathbf{NC_1}, \mathbf{RC_2}\}$. The intracell crossover is applied to two normal cells in two parent individuals, where two examples for the single-point crossover and the revised one-way crossover are presented in Fig. 5 with crossover points $RI = 5$ and $RI = 20$, respectively. After generating two offspring individuals O_1 and O_2 via crossover, the mutation operator is then applied to O_1 and O_2 by applying the single-point mutation and the operator of adding an extra node to the normal cell $\mathbf{NC_1}$. In the single-point crossover, the 7th bit in $\mathbf{NC_1}$ changes from 9 to 3, and the 11th bit changes from 0 to 1. In the operator of adding an extra node, a node $(1, 0, 0, 0, 0, 3)$ is randomly generated to be added to $\mathbf{NC_1}$.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

4) Population Initialization Strategy: The population initialization often has a great influence in convergence and diversity of EAs [25], [27]. Therefore, we design an effective initialization strategy for generating diverse architectures varying from ResNet-like architectures to Inception-like ones. A ResNet-like architecture is constructed by setting the last bit of links in each node to 1 with a high probability, while an Inception-like architecture is constructed by setting the first two bits of links in each node to 1 with a high probability. Hence, it is intuitive to generate diverse architectures by controlling the probability of setting some bits of individuals to 1.

Due to the limit of pages, more detailed procedure of the proposed population initialization strategy can be found in the supplementary material.

IV. Empirical Studies

A. Experiment Settings

1) Benchmark Data Sets: As suggested in most existing ENAS works [13], [14], [20], [21], we conduct the search process of the proposed MFENAS on the CIFAR-10 data set and evaluate the best architecture obtained by the proposed MFENAS on CIFAR-10, CIFAR-100 [34], and ILSVRC 2012 ImageNet [66]. Due to the limit of pages, the details used for each data set are summarized in the supplementary material.

2) Peer Competitors: In order to demonstrate the effectiveness and efficiency of the proposed MFENAS, various state-of-the-art approaches are selected as peer competitors to compare with MFENAS. The selected competitors can be roughly divided into three different types [41]. The first type of competitors is the CNN architectures manually designed, which contains Wide ResNet [67], DenseNet [9], Inception-v1 [10], MobileNet [68], and ShuffleNet [69]. The second type of competitors comprises various non-EA-based NAS approaches, including the RL-based approaches: NASNet [13], MetaQNN [70], Block-QNN-S [71], EAS [72], and E-NAS [14], the gradient-based approaches: DARTS [16] and NAO [17], and the sequential model-based optimization (SMBO [73])-based approach: DDP-Net [74]. The third type of competitors refers to ENAS approaches, including AmoebaNet [31], large-scale evolution [29], hierarchical evolution [30], Genetic-CNN [28], NSGA-Net [20], CNN-GA [21], AE-CNN [32], E2EPP [38], SI-ENAS [41] CARS-E [43], LEMONADE [46], E2NAS-MIG [63], and ACE [62].

Moreover, we also employ another competitor MFENAS(baseline), which is the MFENAS without multifidelity evaluation. In MFENAS(baseline), complete epochs are adopted for neural architecture training.

3) Search Details: Data set Details: According to suggestions in [13], [17], and [21], the search process is executed on the CIFAR-10 data set, where the original training set is divided into a new training set and validation set (90%–10%).

Architecture Details: As suggested in [13], [17], and [20], all generated architectures are set to totally hold 5 ($N = 1$) cells, and the number of filters (channels) in each node is set to 16. The neural architectures will be trained for 25 complete epochs ($S = 25$) by the standard SGD optimizer with momentum, where the learning rate, the momentum rate, and the batch size are set to 0.1, 0.9, and 128, respectively. In addition, a single period cosine decay and L2 weight decay 3×10^{-4} are also utilized.

MFENAS Settings: We adopt the following setting for MFENAS(baseline) and MFENAS: population size Pop = 20, maximum generation Gen = 25, and the range of the number of nodes in initialization $\text{Node}_{\text{range}} = [5, 12]$. The number of fidelities for multifidelity evaluation *MF* in MFENAS is set to 6 to obtain a better tradeoff between the performance and efficiency. The search process of MFENAS(baseline) and MFENAS is executed on one NVIDIA 2080TI GPU.

4) Training Details: After obtaining Pareto-optimal individuals, some promising individuals will be selected to extend the architectures encoded by the individuals and train them on CIFAR-10, CIFAR-100, and ImageNet. Here, all training settings follow the previous NAS approaches [13], [15], [17], [20]. Due to the limit of pages, more detailed settings can be found in the supplementary material.

The training process on both CIFAR-10 and CIFAR-100 is executed on one Tesla P100 GPU, while the training process on ImageNet is executed two Tesla P100 GPUs. The source code of the proposed MFENAS is available at https://github.com/DevilYangS/MFENAS/.

B. Effectiveness of Multifidelity Evaluation

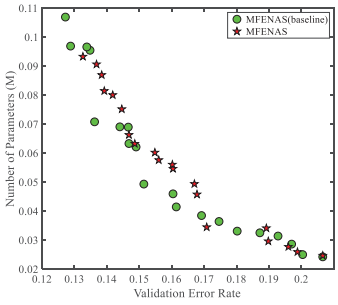
In this section, we will validate the effectiveness of multifidelity evaluation in the proposed MFENAS. First, we verify the performance of the best architectures obtained by MFENAS and MFENAS(baseline) on CIFAR-10, CIFAR-100, and ImageNet. For further validating the effectiveness of our approach, another two search spaces: macro search space and NASNet search space are adopted for MFENAS and MFENAS(baseline). The macro search space is a global search space first proposed in Genetic-CNN [28] and widely used by subsequent approaches [20], [75], while the NASNet search space is another cell-based search space first proposed in NASNet [13] and widely used by subsequent approaches [14], [17], [20], [63]. Table II summarizes comparison results between MFENAS(baseline) and MFENAS in terms of test error rate, the number of model parameters, and search cost, where "-Macro" means the macro search space is adopted and "-NASNet" means the NASNet search space is adopted. As can be seen from Table II, whichever type of search space is used, the search cost of the proposed MFENAS is significantly less than that of MFENAS(baseline), which takes only about 1/4 search cost of MFENAS(baseline). Moreover, the architecture found by the proposed MFENAS achieves a similar error rate to that found by MFENAS(baseline) on each of the three data sets. Particularly, the architecture obtained by the proposed MFENAS has a lower error rate than that obtained by MFENAS(baseline) on the validation data set ImageNet. This validates that the multifidelity evaluation is capable of accelerating ENAS while maintaining high architecture quality. To intuitively verify the final architecture quality obtained by the proposed MFENAS, Fig. 6 presents the validation error rate and the number of parameters in the architectures achieved

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

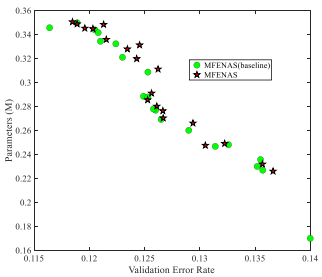
TABLE II
PERFORMANCE COMPARISON OF THE BEST ARCHITECTURE FOUND BY MFENAS(BASELINE) AND MFENAS VALIDATED ON CIFAR-10, CIFAR-100, AND IMAGENET DATA SETS, RESPECTIVELY. NOTE THAT TEST ERROR OF IMAGENET REFERS TO ERROR RATES ON ITS VALIDATION DATA SET

Dataset		Test Error (%)	Parameters (M)	Search Cost
MFENAS(baseline)	CIFAR-10	2.30	2.87	2.55
	CIFAR-100	16.18	2.89	
	ImageNet	26.16	5.79	
	CIFAR-10-Macro	3.34	3.20	2.38
	CIFAR-100-Macro	20.59	3.22	
	CIFAR-10-NASNet	2.72	3.16	2.6
	CIFAR-100-NASNet	16.76	3.20	
MFENAS	CIFAR-10	2.39	2.94	0.6
	CIFAR-100	16.42	2.97	
	ImageNet	26.06	5.98	
	CIFAR-10-Macro	3.46	3.24	0.57
	CIFAR-100-Macro	20.83	3.25	
	CIFAR-10-NASNet	2.85	3.09	0.62
	CIFAR-100-NASNet	17.08	3.13	

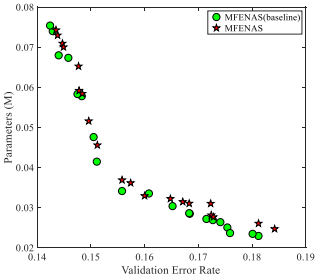
"M" is short for "million" in "Parameters (M)".
"-Macro" means the macro search space in [20], [28] is adopted for MFENAS(baseline) and MFENAS.
"-NASNet" means the NASNet search space in [13], [14] is adopted for MFENAS(baseline) and MFENAS.



(a)



(b)



(c)

Fig. 6. Validation error rate and the number of parameters in the architectures achieved by the proposed MFENAS and MFENAS(baseline). (a) Searched in our proposed cell-based search space. (b) Searched in macro search space. (c) Searched in NASNet search space.

by the proposed MFENAS and MFENAS(baseline). It can be observed from Fig. 6 that the proposed MFENAS finds high-quality architectures that are similar to MFENAS(baseline) in terms of both architecture accuracy and model complexity.

For a deeper insight into the multifidelity evaluation, we investigate the effect of the number of fidelities *MF* in the multifidelity evaluation by recording the reduction ratio of search cost and Kendall's τ value at different values of *MF* ranging from 1 to 12, which is shown in Fig. 7. It is seen from Fig. 7 that the reduction ratio of search cost decreases with the increase of *MF*, indicating that the number of fidelities is effective in controlling the search cost of the proposed MFENAS. Kendall's τ value increases with the increase of *MF*, verifying that the number of fidelities has an influence on the effectiveness of the proposed MFENAS. Therefore, we set *MF* to 6 for balancing the efficiency and effectiveness of the proposed MFENAS, since $MF = 6$ is the best tradeoff between the reduction ratio of search cost and Kendall's τ value. It is also noteworthy that setting *MF* to 6 leads to over 0.7 Kendall's τ value, which is better than Kendall's τ value 0.66 achieved by the best known surrogate-based ENAS approach [38]. This reveals that the multifidelity evaluation is able to balance the effectiveness and efficiency of ENAS by tuning the number of fidelities, which is attributed to the fact that the multifidelity evaluation realizes a tradeoff between evaluation cost and evaluation accuracy in NAS.

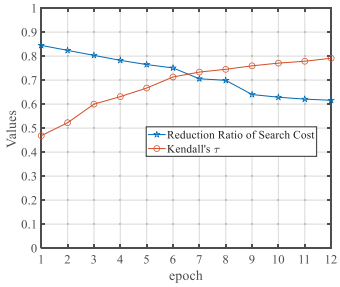


Fig. 7. Reduction ratio of search cost and Kendall's τ with different numbers of fidelities for multifidelity evaluation *MF*.

To summarize, the multifidelity evaluation is effective in accelerating ENAS and maintaining high architecture quality for solving NAS.

C. Competitiveness of the Proposed MFENAS

In this section, we discuss the competitiveness of the proposed MFENAS by comparing it with 22 state-of-the-art NAS approaches. Tables III–V present the architecture error rate, the number of parameters, and the search cost achieved by the proposed MFENAS and existing peer competitors on CIFAR-10, CIFAR-100, and ImageNet.

As presented in Table III, the architecture found by the proposed MFENAS holds a 2.39% test error rate on the CIFAR-10 data set, which is better than test error rates achieved by all the existing peer competitors, including manual designed architectures, non-EA-based NAS approaches, and ENAS approaches. For the search cost, the proposed MFENAS takes only 0.6 GPU days to achieve the architecture with 2.39% test error rate on the CIFAR-10 data set, which is faster than 22 peer competitors but slower than 3 weight-sharing-based NAS approaches. Nevertheless, the search cost of the proposed MFENAS is very approximate to that of 3 weight-sharing-based NAS approaches, which is only 0.3 GPU days larger than the fastest weight-sharing-based approach.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON BETWEEN THE PROPOSED MFENAS AND EXISTING PEER COMPETITORS IN TERMS OF TEST ERROR RATE AND SEARCH COST ON THE CIFAR-10 DATA SET

Peer Competitors	Test Error (%)	Parameters (M)	Search Cost (GPU days)	Search Method
Wide ResNet	4.17	36.5	-	Manual
DenseNet-BC	3.46	25.6	-	Manual
NASNet-A	2.65	3.3	2000	RL
Block-QNN-S	4.38	6.1	96	RL
MetaQNN	6.92	11.2	80	RL
EAS	4.23	23.4	10	RL
E-NAS	2.89	4.6	0.5	RL+weight sharing
DARTS(first order)	3.00	3.3	1.5	Gradient based
DARTS(second order)	2.76	3.3	4	Gradient based
NAO	3.18	10.6	200	Gradient based
NAO+WS	3.53	2.5	0.3	Gradient based+weight sharing
DDP-Net	4.36	11.4	8	SMBO
AmoebaNet-A	3.34	3.2	3150	Evolution
Large-Scale Evolution	5.40	5.4	2750	Evolution
Hierarchical Evolution	3.63	15.7	300	Evolution
Genetic-CNN	7.10	-	17	Evolution
NSGA-Net	2.75	3.3	4	Evolution
CNN-GA	4.78	2.9	35	Evolution
AE-CNN	4.30	2.0	27	Evolution
E2EPP	5.30	-	8.5	Evolution
SI-ENAS	4.07	-	1.8	Evolution
CARS-E	2.86	3.0	0.4	Evolution+weight sharing
LEMONADE	3.05	4.7	80	Evolution
E2NAS-MIG	4.8	3.8	15	Evolution
ACE	2.9	2.4	1.5	Evolution
MFENAS	**2.39**	2.94	0.6	Evolution

The results of comparison approaches are directly obtained from their original papers.
'-' represents that the corresponding result is not publicly available.

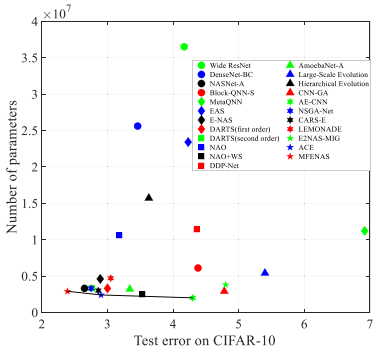


Fig. 8. Test error and number of parameters of the architectures found by human experts and NAS approaches on CIFAR-10, where the architectures having the best tradeoffs between test error and complexity are connected.

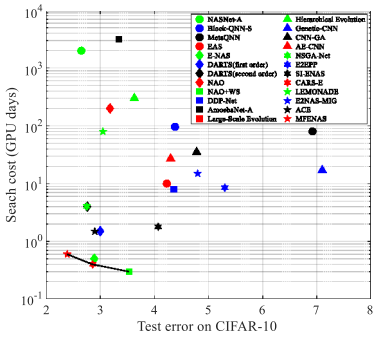


Fig. 9. Test error and search cost of final architectures found by NAS approaches on CIFAR-10, where the architectures having the best tradeoffs between test error and search cost are connected.

The proposed MFENAS outperforms 22 peer competitors and is competitive to 3 weight-sharing-based NAS approaches in terms of architecture performance and search cost. Hence, the proposed MFENAS is overall superior over or competitive to state-of-the-art NAS approaches in terms of effectiveness and efficiency on CIFAR-10.

Fig. 8 depicts the test error and number of model parameters of the architectures listed in Table III. It can be observed that three architectures have the best tradeoffs between test error and complexity, where they are obtained by AE-CNN, ACE, and our proposed MFENAS. It is worth noting that the architecture found by MFENAS holds the best test error 2.39%, which has a 1.91% test accuracy improvement at a cost of extra 0.94M parameters compared to AE-CNN. Besides, Fig. 9 plots the test error and search cost of the architectures found by NAS approaches, where the best tradeoff architecture found by MFENAS has a little larger search cost but lower test error that the other tradeoff architectures. In summary, the proposed MFENAS can strike a balance among the performance, the model complexity, and the search cost. For deeper insight into the effectiveness of the proposed MFENAS, Fig. 10

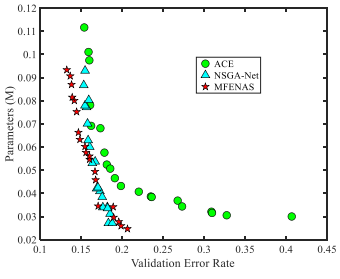


Fig. 10. Pareto fronts achieved by ACE, NSGA-Net, and the proposed MFENAS.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPARISON BETWEEN THE PROPOSED MFENAS AND EXISTING PEER COMPETITORS IN TERMS OF TEST ERROR RATE AND SEARCH COST ON THE CIFAR-100 DATA SET

Peer Competitors	Test Error (%)	Parameters (M)	Search Cost (GPU days)	Search Method
Wide ResNet	20.50	36.5	-	Manual
DenseNet-BC	17.18	25.6	-	Manual
NASNet-A	16.58	3.3	2000	RL
Block-QNN-S	20.65	6.1	96	RL
MetaQNN	27.14	11.2	80	RL
E-NAS	17.27	4.6	0.5	RL+weight sharing
NAO	15.67	10.8	200	Gradient based
AmoebaNet-B	15.80	3.2	3150	Evolution
Large-Scale Evolution	23.00	40.4	2750	Evolution
Genetic-CNN	29.03	-	17	Evolution
NSGA-Net	20.74	3.3	8	Evolution
CNN-GA	20.53	4.1	40	Evolution
AE-CNN	20.85	5.4	36	Evolution
E2EPP	22.02	-	8.5	Evolution
SI-ENAS	18.64	-	1.8	Evolution
ACE	18.84	2.4	1.5	Evolution
MFENAS	**16.42**	2.97	0.6	Evolution

Note that only 16 peer competitors are adopted for comparison on CIFAR-100 since the architecture quality achieved by the remaining peer competitors on CIFAR-100 is not publicly available.

TABLE V
COMPARISON BETWEEN THE PROPOSED MFENAS AND EXISTING PEER COMPETITORS IN TERMS OF ERROR RATE AND SEARCH COST ON THE IMAGENET DATA SET

Peer Competitors	Error (%) Top-1	Error (%) Top-5	Params (M)	Search Cost (GPU days)	Search Method
Inception-v1	30.20	10.10	6.6	-	Manual
MobileNet	29.40	10.50	4.2	-	Manual
ShuffleNet (v1)	29.10	10.20	5	-	Manual
NASNet-A	26.00	8.4	5.3	2000	RL
NASNet-B	27.2	8.7	5.3	2000	RL
NASNet-C	27.5	9	4.9	2000	RL
DARTS	26.70	8.70	4.7	4	Gradient based
NAO	25.70	8.20	11.4	200	Gradient based
DDP-Net	25.98	8.21	7.16	8	SMBO
AmoebaNet-A	25.50	8.00	5.1	3150	Evolution
AmoebaNet-B	26.00	8.50	5.3	3150	Evolution
AmoebaNet-C	24.30	7.60	6.4	3150	Evolution
Genetic-CNN	27.87	9.74	156.0	17	Evolution
CARS-E	26.30	8.40	4.4	0.4	Evolution+ weight sharing
LEMONADE	28.3	9.6	6.0	80	Evolution
MFENAS	**26.06**	**8.18**	**5.98**	**0.6**	Evolution

Note that the table only compares the peer competitors whose architecture quality is publicly available.

compares the Pareto fronts obtained by three multiobjective NAS methods: the proposed MFENAS, ACE, and NSGA-Net, considering that only the source codes of NSGA-Net and ACE are available in the compared multiobjective NAS methods. As can be observed from Fig. 10, the Pareto front of the proposed MFENAS is significantly better than that of ACE and slightly better than that of NSGA-Net.

To further investigate the competitiveness of the proposed MFENAS, we transfer the best architecture found by MFENAS from CIFAR-10 with ten classes to CIFAR-100 with 100 classes. Table IV presents the comparison results between the proposed MFNEAS and existing peer competitors on CIFAR-100. As can be observed from Table IV, the transferred architecture of the proposed MFENAS achieves a 16.42% test error rate on CIFAR-100, which performs better than 14 peer competitors but worse than 2 peer competitors (i.e., NAO and AmoebaNet-B). The reason may lie in the fact that NAO and AmoebaNet-B hold more model parameters than the proposed MFENAS. However, the search cost of the proposed MFENAS is significantly lower than that of NAO and AmoebaNet-B on CIFAR-100, indicating that the proposed MFENAS is competitive to NAO and AmoebaNet-B when considering both architecture performance and search cost. Furthermore, the search cost of the proposed MFENAS is lower than 15 peer competitors but higher than one competitor E-NAS. The proposed MFENAS still can be regarded as competitive to E-NAS due to its superiority over E-NAS in terms of architecture performance. Therefore, the proposed MFENAS overall has better or competitive performance compared to state-of-the-art NAS approaches in terms of effectiveness and efficiency on CIFAR-100.

In addition to CIFAR-10 and CIFAR-100, we also evaluate the performance of the proposed MFENAS on a more challenging data set ImageNet, which holds much more classes, much larger image size, and much more images compared to CIFAR-10 and CIFAR-100. Table V provides the comparison results between the proposed MFENAS and peer competitors on ImageNet, where the best architecture obtained by the proposed MFENAS is transferred from CIFAR-10 to ImageNet. It can be seen from Table V that the transferred architecture of the proposed MFENAS holds a 26.06% error rate, which is lower than 9 peer competitors but higher than NASNet-A, NAO, DDP-Net, and AmoebaNet. Nevertheless, it can still be drawn from Table V that the proposed MFENAS is competitive to NASNet-A, NAO, DDP-Net, and AmoebaNet on ImageNet, since the search cost of the proposed MFENAS is significantly lower than that of NASNet-A, NAO, DDP-Net, and AmoebaNet. More specifically, the search cost of the proposed MFENAS is lower than all the competitors on ImageNet. Consequently, the performance of the proposed MFENAS overall is better than or competitive to state-of-the-art peer competitors in terms of effectiveness and efficiency on ImageNet.

For summary, the proposed MFENAS is demonstrated to be superior over or competitive to state-of-the-art NAS approaches in terms of effectiveness and efficiency.

D. Discussion on Details of the Proposed MFENAS

In this section, we first verify the effectiveness of the population initialization strategy suggested for the proposed MFENAS. Then, we analyze the efficacy of the architecture search space suggested in the proposed MFENAS.

Fig. 11 presents the initial populations and the convergence profiles of hypervolume (HV) obtained by random initialization, the proposed population initialization, and the combination of two initialization strategies (denoted as *Initialization* + *Random*) under the MFENAS(baseline) framework, where HV measures convergence and diversity of a population and a large HV value indicates a good convergence and diversity [76]. Note that the MFENAS(baseline) is

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

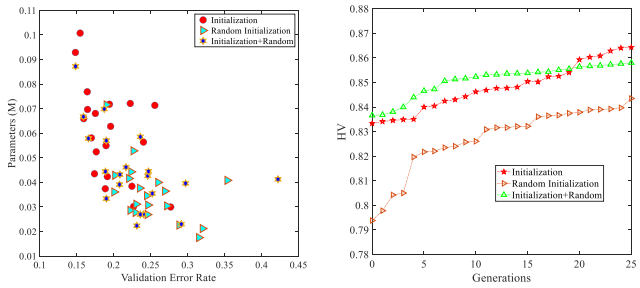


Fig. 11. Comparison between random initialization, the proposed population initialization, and the combination of them under the MFENAS(baseline) framework. (a) Initial populations obtained by random initialization, the proposed population initialization strategy, and the combination of them. (b) Convergence profiles of HV obtained by random initialization, the proposed population initialization strategy, and the combination of them.

TABLE VI
EFFECTIVENESS VALIDATION OF ARCHITECTURE SEARCH SPACE BY COMPARING MFENAS(BASELINE) AND NSGA-NET ON CIFAR-10 AND CIFAR-100 DATA SETS

	Dataset	Test Error (%)	Parameters (M)
CIFAR-10	NSGA-Net	2.75	3.3
	MFENAS(baseline)	**2.30**	**2.87**
CIFAR-100	NSGA-Net	20.74	3.3
	MFENAS(baseline)	**16.18**	**2.89**

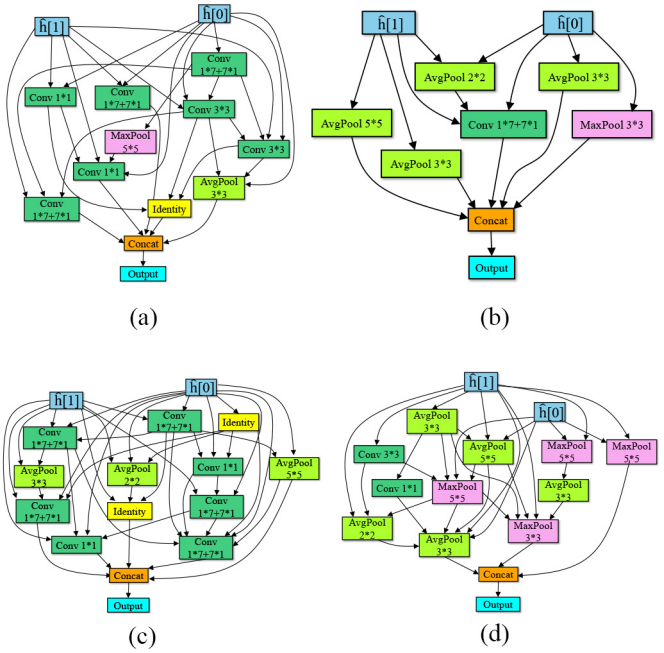


Fig. 12. Best architecture obtained by MFENAS(baseline) and MFENAS, where they hold similar good performance on three data sets but having quite different structure especially for their reduction cells. (a) Normal cell found by MFENAS(baseline). (b) Reduction cell found by MFENAS(baseline). (c) Normal cell found by MFENAS. (d) Reduction cell found by MFENAS.

adopted as the framework to eliminate the effect of multifidelity evaluation in the performance of the three initialization strategies. As can be observed from Fig. 11, the diversity of the initial population obtained by the suggested population initialization is sightly worse than that of *Initialization + Random* but better than that of random initialization. Besides, the suggested population initialization leads to better convergence in MFENAS(baseline) compared to the other two initialization strategies. Therefore, the suggested initialization strategy is effective in strengthening the performance of the proposed MFENAS by generating diverse architectures in the initialization of MFENAS.

To investigate the effect of the suggested architecture search space in the proposed MFENAS, Table VI compares the best architecture found by MFENAS(baseline) with that found by NSGA-Net, which is a fair comparison since the two approaches have the same number of sampled architectures. MFENAS(baseline) shares the same architecture search space with the proposed MFENAS as suggested in Section III-C1, while NSGA-Net utilizes the NASNet search space. It can be seen from Table VI that MFENAS(baseline) can find a higher quality architecture than NSGA-Net. Specifically, Fig. 12 presents the best architecture obtained by the proposed MFENAS and that obtained by MFENAS(baseline). The two architectures obtained by the proposed MFENAS and MFENAS(baseline) are quite different from each other under the suggested search space, while under NASNet search space, the best architecture obtained by different NAS approaches is often similar to each other [77]. As a result, the suggested search space facilitates NAS approaches in generating more diverse and flexible architectures than NASNet search space for solving NAS problems.

V. CONCLUSION AND FUTURE WORK

In this article, we have developed an accelerated ENAS approach via multifidelity evaluation named MFENAS, to achieve high-quality neural architectures at a low computational cost in NAS. Empirical results on CIFAR-10 have shown that the architecture found by the proposed MFENAS achieves a 2.39% test error rate at the cost of only 0.6 GPU days on one NVIDIA 2080TI GPU. Compared with 22 state-of-the-art NAS approaches, the proposed MFENAS has exhibited superior performance in terms of both computational cost and architecture quality on CIFAR-10. The architecture transferred to CIFAR-100 and ImageNet has also shown competitive performance to the architectures obtained by existing approaches.

In this article, the proposed MFENAS mainly concentrates on efficiently finding high-quality architectures, where the vulnerability of architectures is not considered. The vulnerability of architectures plays an important role in defensing adversarial attack [77]. Hence, in the future, we would like to develop new ENAS approaches to find robust architectures against various adversarial attacks. Moreover, the binary NN is characterized with a low memory saving and a low inference latency [78], but few NAS studies have been reported on searching for binary neural architectures. Therefore, it is also interesting to design new ENAS approaches for searching high-quality binary neural architectures.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] M. Niepert, M. Ahmed, and K. Kutzkov, "Learning convolutional neural networks for graphs," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 2014–2023.

[2] W. Liu *et al.*, "SSD: Single shot multibox detector," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 21–37.

[3] O. Abdel-Hamid, A.-R. Mohamed, H. Jiang, L. Deng, G. Penn, and D. Yu, "Convolutional neural networks for speech recognition," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 22, no. 10, pp. 1533–1545, Oct. 2014.

[4] T. Young, D. Hazarika, S. Poria, and E. Cambria, "Recent trends in deep learning based natural language processing [review article]," *IEEE Comput. Intell. Mag.*, vol. 13, no. 3, pp. 55–75, Aug. 2018.

[5] A. K. Verma and T. N. Singh, "Comparative study of cognitive systems for ground vibration measurements," *Neural Comput. Appl.*, vol. 22, no. 1, pp. 341–350, 2013.

[6] G.-H. Lee, Y.-D. Lee, and I.-S. Koo, "Convolution neural network-based spectrum sensing for cognitive radio systems using USRP with GNU radio," in *Proc. 10th Int. Conf. Ubiquitous Future Netw. (ICUFN)*, 2018, pp. 862–864.

[7] S. Tripathi, C. Kundu, O. A. Dobre, A. Bansal, and M. F. Flanagan, "Recurrent neural network assisted transmitter selection for secrecy in cognitive radio network," in *Proc. IEEE Global Commun. Conf.*, 2020, pp. 1–6.

[8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[9] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 4700–4708.

[10] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1–9.

[11] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.

[12] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," 2016, *arXiv:1611.01578*.

[13] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8697–8710.

[14] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4095–4104.

[15] Y. Chen *et al.*, "Renas: Reinforced evolutionary neural architecture search," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4787–4796.

[16] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–13.

[17] R. Luo, F. Tian, T. Qin, E. Chen, and T.-Y. Liu, "Neural architecture optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 7816–7827.

[18] K. Kandasamy, W. Neiswanger, J. Schneider, B. Poczos, and E. P. Xing, "Neural architecture search with bayesian optimisation and optimal transport," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 2016–2025.

[19] H. Jin, Q. Song, and X. Hu, "Auto-keras: An efficient neural architecture search system," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2019, pp. 1946–1956.

[20] Z. Lu *et al.*, "NSGA-Net: Neural architecture search using multi-objective genetic algorithm," in *Proc. Genet. Evol. Comput. Conf.*, 2019, pp. 419–427.

[21] Y. Sun, B. Xue, M. Zhang, G. G. Yen, and J. Lv, "Automatically designing CNN architectures using the genetic algorithm for image classification," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3840–3854, Sep. 2020.

[22] Q. Song, D. Cheng, H. Zhou, J. Yang, Y. Tian, and X. Hu, "Towards automated neural interaction discovery for click-through rate prediction," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2020, pp. 945–955.

[23] Z. Lu *et al.*, "Multiobjective evolutionary design of deep convolutional neural networks for image classification," *IEEE Trans. Evol. Comput.*, vol. 25, no. 2, pp. 277–291, Apr. 2021.

[24] C. Prins, "A simple and effective evolutionary algorithm for the vehicle routing problem," *Comput. Oper. Res.*, vol. 31, no. 12, pp. 1985–2002, 2004.

[25] Y. Tian, S. Yang, and X. Zhang, "An evolutionary multiobjective optimization based fuzzy method for overlapping community detection," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 11, pp. 2841–2855, Nov. 2020.

[26] X. Xiang, Y. Tian, J. Xiao, and X. Zhang, "A clustering-based surrogate-assisted multiobjective evolutionary algorithm for shelter location problem under uncertainty of road networks," *IEEE Trans. Ind. Informat.*, vol. 16, no. 12, pp. 7544–7555, Dec. 2020.

[27] L. Zhang, S. Yang, X. Wu, F. Cheng, Y. Xie, and Z. Lin, "An indexed set representation based multi-objective evolutionary approach for mining diversified top-k high utility patterns," *Eng. Appl. Artif. Intell.*, vol. 77, pp. 9–20, Jan. 2019.

[28] L. Xie and A. Yuille, "Genetic CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 1379–1388.

[29] E. Real *et al.*, "Large-scale evolution of image classifiers," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2902–2911.

[30] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, and K. Kavukcuoglu, "Hierarchical representations for efficient architecture search," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–13.

[31] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 4780–4789.

[32] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Completely automated CNN architecture design based on blocks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 4, pp. 1242–1254, Apr. 2020.

[33] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Evolving deep convolutional neural networks for image classification," *IEEE Trans. Evol. Comput.*, vol. 24, no. 2, pp. 394–407, Apr. 2020.

[34] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Rep. TR-2009, 2009.

[35] K. Swersky, J. Snoek, and R. P. Adams, "Freeze-thaw Bayesian optimization," 2014, *arXiv:1406.3896*.

[36] B. Baker, O. Gupta, R. Raskar, and N. Naik, "Accelerating neural architecture search using performance prediction," 2017, *arXiv:1705.10823*.

[37] B. Deng, J. Yan, and D. Lin, "Peephole: Predicting network performance before training," 2017, *arXiv:1712.03351*.

[38] Y. Sun, H. Wang, B. Xue, Y. Jin, G. G. Yen, and M. Zhang, "Surrogate-assisted evolutionary deep learning using an end-to-end random forest-based performance predictor," *IEEE Trans. Evol. Comput.*, vol. 24, no. 2, pp. 350–364, Apr. 2020.

[39] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "A particle swarm optimization-based flexible convolutional autoencoder for image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 8, pp. 2295–2309, Aug. 2019.

[40] F. Assunção, N. Lourenço, P. Machado, and B. Ribeiro, "Fast denser: Efficient deep neuroevolution," in *Proc. Eur. Conf. Genet. Program.*, 2019, pp. 197–212.

[41] H. Zhang, Y. Jin, R. Cheng, and K. Hao, "Sampled training and node inheritance for fast evolutionary neural architecture search," 2020, *arXiv:2003.11613*.

[42] P. Liu, M. D. El Basha, Y. Li, Y. Xiao, P. C. Sanelli, and R. Fang, "Deep evolutionary networks with expedited genetic algorithms for medical image denoising," *Med. Image Anal.*, vol. 54, pp. 306–315, May 2019.

[43] Z. Yang *et al.*, "CARS: Continuous evolution for efficient neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1829–1838.

[44] H. Wang, Y. Jin, and J. Doherty, "A generic test suite for evolutionary multifidelity optimization," *IEEE Trans. Evol. Comput.*, vol. 22, no. 6, pp. 836–850, Dec. 2018.

[45] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[46] T. Elsken, J. H. Metzen, and F. Hutter, "Efficient multi-objective neural architecture search via lamarckian evolution," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–23.

[47] M. Laumanns and J. Ocenasek, "Bayesian optimization algorithms for multi-objective optimization," in *Proc. Int. Conf. Parallel Problem Solving Nature*, 2002, pp. 298–307.

[48] S. Bandyopadhyay, S. Saha, U. Maulik, and K. Deb, "A simulated annealing-based multiobjective optimization algorithm: AMOSA," *IEEE Trans. Evol. Comput.*, vol. 12, no. 3, pp. 269–283, Jun. 2008.

[49] J. Pacheco and R. Martí, "Tabu search for a multi-objective routing problem," *J. Oper. Res. Soc.*, vol. 57, no. 1, pp. 29–37, 2006.

[50] C. A. Coello Coello, "Evolutionary multi-objective optimization: A historical view of the field," *IEEE Comput. Intell. Mag.*, vol. 1, no. 1, pp. 28–36, Feb. 2006.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.

[51] H. Zhang, Q. Zhang, L. Ma, Z. Zhang, and Y. Liu, "A hybrid ant colony optimization algorithm for a multi-objective vehicle routing problem with flexible time windows," *Inf. Sci.*, vol. 490, pp. 166–190, Jul. 2019.

[52] Z. Lu, K. Deb, E. Goodman, W. Banzhaf, and V. N. Boddeti, "NSGANetV2: Evolutionary multi-objective surrogate-assisted neural architecture search," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 35–51.

[53] M. Suganuma, M. Kobayashi, S. Shirakawa, and T. Nagao, "Evolution of deep convolutional neural networks using cartesian genetic programming," *Evol. Comput.*, vol. 28, no. 1, pp. 141–163, 2020.

[54] F. Assunção *et al.*, "Automatic design of artificial neural networks for gamma-ray detection," *IEEE Access*, vol. 7, pp. 110531–110540, 2019.

[55] D. R. So, C. Liang, and Q. V. Le, "The evolved transformer," 2019, *arXiv:1901.11117*.

[56] Y. Liu, Y. Sun, B. Xue, M. Zhang, and G. Yen, "A survey on evolutionary neural architecture search," 2020, *arXiv:2008.10937*.

[57] Z. Fan, J. Wei, G. Zhu, J. Mo, and W. Li, "Evolutionary neural architecture search for retinal vessel segmentation," 2020, *arXiv:2001.06678*.

[58] F. M. Johner and J. Wassner, "Efficient evolutionary architecture search for CNN optimization on GTSRB," in *Proc. 18th IEEE Int. Conf. Mach. Learn. And Appl. (ICMLA)*, 2019, pp. 56–61.

[59] H. Zhang, S. Kiranyaz, and M. Gabbouj, "Finding better topologies for deep convolutional neural networks by evolution," 2018, *arXiv:1809.03242*.

[60] P. K. Sen, "Estimates of the regression coefficient based on Kendall's tau," *J. Amer. Stat. Assoc.*, vol. 63, no. 324, pp. 1379–1389, 1968.

[61] J. Ren *et al.*, "Application of multifidelity optimization techniques to benchmark aerodynamic design problems," in *Proc. 54th AIAA Aerosp. Sci. Meeting*, 2016, p. 1542.

[62] Y. Tian, S. Peng, S. Yang, X. Zhang, K. C. Tan, and Y. Jin, "Action command encoding for surrogate assisted neural architecture search," *IEEE Trans. Cogn. Devel. Syst.*, early access, Aug. 25, 2021, doi: 10.1109/TCDS.2021.3107555.

[63] C. He, H. Tan, S. Huang, and R. Cheng, "Efficient evolutionary neural architecture search by modular inheritable crossover," *Swarm Evol. Comput.*, vol. 64, Jul. 2021, Art. no. 100894.

[64] C. A. C. Coello, G. B. Lamont, and D. A. Van Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, vol. 5. Boston, MA, USA: Springer, 2007.

[65] Y. Tian, X. Zhang, C. Wang, and Y. Jin, "An evolutionary algorithm for large-scale sparse multi-objective optimization problems," *IEEE Trans. Evol. Comput.*, vol. 24, no. 2, pp. 380–393, Apr. 2020.

[66] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.

[67] S. Zagoruyko and N. Komodakis, "Wide residual networks," 2016, *arXiv:1605.07146*.

[68] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[69] X. Zhang, X. Zhou, M. Lin, and J. Sun, "ShuffleNet: An extremely efficient convolutional neural network for mobile devices," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 6848–6856.

[70] B. Baker, O. Gupta, N. Naik, and R. Raskar, "Designing neural network architectures using reinforcement learning," 2016, *arXiv:1611.02167*.

[71] Z. Zhong, J. Yan, W. Wu, J. Shao, and C.-L. Liu, "Practical block-wise neural network architecture generation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2423–2432.

[72] H. Cai, T. Chen, W. Zhang, Y. Yu, and J. Wang, "Efficient architecture search by network transformation," in *Proc. AAAI*, 2018, p. 340.

[73] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *Proc. Int. Conf. Learn. Intell. Optim.*, 2011, pp. 507–523.

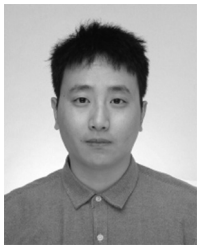
[74] J.-D. Dong, A.-C. Cheng, D.-C. Juan, W. Wei, and M. Sun, "DPP-Net: Device-aware progressive search for Pareto-optimal neural architectures," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 517–531.

[75] M. Wistuba, A. Rawat, and T. Pedapati, "A survey on neural architecture search," 2019, *arXiv:1905.01392*.

[76] L. While, P. Hingston, L. Barone, and S. Huband, "A faster algorithm for calculating hypervolume," *IEEE Trans. Evol. Comput.*, vol. 10, no. 1, pp. 29–38, Feb. 2006.

[77] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 55, pp. 1–21, 2019.

[78] K. P. Singh, D. Kim, and J. Choi, "Learning architectures for binary networks," 2020, *arXiv:2002.06963*.



Shangshang Yang received the B.Sc. degree from Anhui University, Hefei, China, in 2017, where he is currently pursuing the Ph.D. degree with the School of Computer Science and Technology.

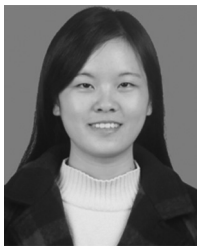
His current research interests include evolutionary multiobjective optimization and neural architecture search.



Ye Tian (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees from Anhui University, Hefei, China, in 2012, 2015, and 2018, respectively.

He is currently an Associate Professor with the Institute of Physical Science and Information Technology, Anhui University. His current research interests include evolutionary computation and its applications.

Dr. Tian is the recipient of the 2018 and 2021 IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION Outstanding Paper Award, the 2020 *IEEE Computational Intelligence Magazine* Outstanding Paper Award, and the 2022 IEEE Computational Intelligence Society Outstanding Ph.D. Dissertation Award.



Xiaoshu Xiang received the B.Eng. degree from Anhui Normal University, Wuhu, China, in 2015, and the Ph.D. degree from Anhui University, Hefei, China, in 2020.

She is currently a Lecturer with the Institute of Physical Science and Information Technology, Anhui University. Her research interests include evolutionary computation, dynamic optimization, and logistic scheduling.



Shichen Peng received the B.Sc. degree from Anhui University, Hefei, China, in 2018, where he is currently pursuing the M.Sc. degree with the School of Computer Science and Technology.

His current research interests include evolutionary optimization and deep learning.



Xingyi Zhang (Senior Member, IEEE) received the B.Sc. degree from Fuyang Normal College, Fuyang, China, in 2003, and the M.Sc. and Ph.D. degrees from Huazhong University of Science and Technology, Wuhan, China, in 2006 and 2009, respectively.

He is currently a Professor with the School of Artificial Intelligence, Anhui University, Hefei, China. His current research interests include unconventional models and algorithms of computation, multiobjective optimization, and membrane computing.

Prof. Zhang is the recipient of the 2018 and 2021 IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION Outstanding Paper Award and the 2020 *IEEE Computational Intelligence Magazine* Outstanding Paper Award.

Authorized licensed use limited to: Anhui University. Downloaded on October 25,2026 at 13:54:05 UTC from IEEE Xplore. Restrictions apply.